

-Applied Artificial Intelligence Project-

Presented to:

Doç. Dr. Emine Uçar

Presented by:

Kurtay Emin Tuğrul – 221005062

Fahrettin Çevik – 221005028

Burcu Bağcı – 211005012

CONTENTS

INTRODUCTION AND SCOPE OF THE PROJECT	3
EVALUATION OF DATASET.....	3
DATASET DESCRIPTIONS	3
BRIEF OF DATASET COLUMNS (FEATURES)	4
INVENTORY.....	4
<i>Libraries Used:</i>	4
<i>Functions and methods we used:</i>	5
CODING PROCESS AND NECESSARY EXPLANATIONS, CAUSE AND EFFECT RELATIONSHIPS	5
ANALYSIS OF DATA:	5
NULL CHECK	6
BALANCE CHECK	6
SOLVING THE BALANCE PROBLEM.....	6
DEFINING INPUTS AND OUTPUTS	7
MAKING DATA SUITABLE FOR LEARNING (SCALING)	7
DEFINITION OF VARIABLES:	8
RESULTS OBTAINED, SCORES AND THEIR EVALUATION:	9
RESULT WITH GINI:	9
RESULT WITH ENTROPY:	9
EVALUATION OF ACCURACY SCORES AND PURPOSE OF USING GINI AND ENTROPY	9
CONFUSION MATRIX RESULTS:	10
WHAT ELSE COULD HAVE BEEN DONE?	11
CONCULISON	11

Introduction and Scope of the project

Our scope in this Project; using the machine learning dynamics we learned in the course. The content of the report will cover the steps we used in the machine learning we prepared, the reasons and results of the decisions we made. In addition, there will be explanations of the dataset, variables and other details used.

Evaluation of Dataset

Dataset Descriptions

The dataset we used is the Credit Card Fraud dataset uploaded by Dhanush Narayan on Kaggle 3 years ago. link to the dataset:

<https://www.kaggle.com/datasets/dhanushnarayanar/credit-card-fraud/data>

The reasons for choosing this dataset:

- 1- The content of the dataset is different, unique and interesting is the first factor that make this choice.
 - 2- The reality of the data types (columns) used in the dataset.
 - 3- Preferred by many people in Kaggle and many people worked on this dataset.
 - 4- Dataset contains data that is not difficult to deal with and work with. For example, values in columns with yes/no answers were already converted to binary (1/0) values.
-
- Decision Tree models are used in this code because in binary classification tasks such as the fraud detection problem, decision trees provide high accuracy by easily learning complex data patterns and producing interpretable rules, provide fast, efficient results, especially on balanced data sets.

Brief of Dataset Columns (Features)

The dataset contains 8 different columns suitable for real life.

Column (Feature) Name	Value Type	Brief explanation
distance_from_home	Numerical Value	The distance between the transaction location and the user's home.
distance_from_last_transaction	Numerical Value	The distance between this transaction and the previous one.
ratio_to_median_purchase_price	Numerical Value	Ratio of this transaction's amount to the user's median purchase amount.
repeat_retailer	0 or 1 (Binary)	User shopped at this retailer before?
used_chip	0 or 1 (Binary)	Card's chip used during the transaction?
used_pin_number	0 or 1 (Binary)	PIN number entered?
online_order	0 or 1 (Binary)	Was the transaction made online?
fraud	0 or 1 (Binary)	Target, Transaction is fraudulent?

Inventory

In this part, we are going to explain which libraries and functions we used, why we used them and their explanation?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import StandardScaler
from sklearn.utils import resample
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn import tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, accuracy_score
```

Libraries Used:

Numpy: Numerical library. Imported as np.

Pandas : Data manipulation and analysis library imported as pd.

Matplotlib.pyplot : module for plotting and visualization, imported as plt.

Sklearn: scikit-learn library for machine learning.

Functions and methods we used:

From sklearn.compose import make_column_transfer :

- ❖ We used this line to apply transformations to features with numeric values, because during reforestation we needed to work with standardized data instead of all values.

From sklearn.preprocessing import StandardScaler :

- ❖ This line standardize numerical features, which improves the performance of machine learning models like decision trees on features with different scales.

From sklearn.utils import resample:

- ❖ Addressing class imbalance in the fraud detection dataset to improve model fairness.

From sklearn.model_selection import train_test_split :

- ❖ To split the dataset into training and test sets.

From sklearn.tree import DecisionTreeClassifier:

- ❖ To build a decision tree model selected to classify fraud and non-fraud cases.

Coding process and necessary explanations, cause and effect relationships

We decided to explain the lines of code sequentially in order to explain the other elements used, so that we can explain our operations in a cause and effect relationship.

Analysis of data:

1- We get results from dataset.shape and dataset.head to see the number of data and to learn the operations we will apply to the columns.

As a result, we decided that there are 3 features that we need to implement in StandardScaler

```
print("Size of Dataset:", dataset.shape)
print("-----")
dataset.head(10)
```

Size of Dataset: (1000000, 8)

	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_pr
0	57.877857	0.311140	1.945

Null Check

2- Since there are exactly one million rows, we could not check the dataset visually, so we checked for null values with this line. And we obtained that there are no null values

```
[ ] print(dataset.isnull().sum())
```

distance_from_home	0
distance_from_last_transaction	0
ratio_to_median_purchase_price	0
repeat_retailer	0
used_chip	0
used_pin_number	0
online_order	0
fraud	0
dtype: int64	

Balance Check

3- We decided to take all our precautions before starting the machine learning so as not to fall into repetitive operations, We decided to look at the distributions of our outputs and as a result our dataset is extremely unbalanced. Next step we will show how we avoided this.

```
print("balance test:\n", dataset['fraud'].value_counts())
```

balance test:	
fraud	
0.0	912597
1.0	87403
Name: count, dtype: int64	

Solving the balance problem

4- The outputs are extremely unbalanced. we decided to equalize the number of outputs to avoid overfitting. As a result of the operations: Instead of adding artificial data to output type '1', we decided to equalize by deleting data from output type '0', and for this we did the following part: We have used the resample function and synchronized the number of samples to the other output type with len(data_1).

```
data_0 = dataset[dataset.fraud == 0]
data_1 = dataset[dataset.fraud == 1]

data_0_downsampled = resample(data_0, replace=False, n_samples=len(data_1), random_state=42)

balanced_dataset = pd.concat([data_0_downsampled, data_1])

balanced_dataset = balanced_dataset.sample(frac=1, random_state=42).reset_index(drop=True)

print(balanced_dataset['fraud'].value_counts())
```

fraud	
0.0	87403
1.0	87403
Name: count, dtype: int64	

Defining Inputs and Outputs

5- We were ready to start machine learning. We started by separating the inputs and outputs. We separated the fraud feature in the balanced dataset as output and we gave the remaining features to X as input.

```
 X = balanced_dataset.drop("fraud", axis=1)  
Y = balanced_dataset["fraud"]
```

Making data suitable for learning (scaling)

6- We needed to categorize the features containing numeric values so that we could get consistent results because we are doing classification. so we took our 3 features into the StandardScaler function and separated the rest.

```
 column_trans = make_column_transformer(  
    (StandardScaler(), ['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_purchase_price']),  
    remainder='passthrough'  
)  
  
X = column_trans.fit_transform(X)  
  
print(X.shape)
```

Note: we didn't use OneHotEncoder because features already have binary representation instead of YES/NO. But we have added the remainder='passthrough' line to our code so these binary features do not get into the StandardScaler process and lose their nature. So we have specified the columns in dataset that need to be standardized in the line of code and used the 'passthrough' command to allocate the rest.

7-In this step, we divided our data into test and train as 30/70 and wrote the necessary codes for learning operations. the results we obtained will be evaluated in the next section.

```
 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size= 0.3 , random_state=42)  
  
[ ] model_entropy= DecisionTreeClassifier(criterion = 'entropy', random_state=42)  
    model_gini = DecisionTreeClassifier ( criterion = 'gini', max_depth= 5, random_state=42)  
  
[ ] model_gini.fit(X_train, Y_train)  
    predicts= model_gini.predict(X_test)  
  
print("Accuracy score with test set = ", model_gini.score(X_test, Y_test))  
print("Accuracy score with train set = ", model_gini.score(X_train, Y_train))
```

```
 model_entropy.fit(X_train, Y_train)  
  
predicts= model_entropy.predict(X_test)  
  
print("Accuracy score with test set = ", model_entropy.score(X_test, Y_test))
```

Definition of Variables:

Name of Variable	Short description
dataset	Dataset loaded with Pandas from CSV file
Data_0	Data containing rows where fraud = 0
Data_1	Data containing rows where fraud = 1
Data_0_downsampled	Fixed version of data_0 (downsampled)
Balanced_dataset	Balanced dataset after resample
X	Features which using for input
Y	Fraud feature that using for output
X_train	Training set for inputs
Y_train	Training set for outputs
X_test	Test set for inputs
Y_test	Test set for outputs
Model_gini	Object of DecisionTreeClassifier with Gini criterion
Model_entropy	Object of DecisionTreeClassifier with entropy criterion
Predicts	Predictions from the model
Feature_names	List of feature names for Tree and branching visualization
Gini_tree_text	Text representation of the Gini

Results obtained, scores and their evaluation:

explanations will be made after the results obtained are shown.

Result with Gini:

```
model_gini.fit(X_train, Y_train)
predicts= model_gini.predict(X_test)

print("Accuracy score with test set = ", model_gini.score(X_test, Y_test))
print("Accuracy score with train set = ", model_gini.score(X_train, Y_train))
```

Accuracy score with test set = 0.9986461233362572
Accuracy score with train set = 0.9983246706547677

Result with Entropy:

```
model_entropy.fit(X_train, Y_train)

predicts= model_entropy.predict(X_test)

print("Accuracy score with test set = ", model_entropy.score(X_test, Y_test))
```

Accuracy score with test set = 0.999885587887571

Evaluation of Accuracy Scores and Purpose of Using Gini and Entropy

- ✓ The model achieved an accuracy score of 99.864% on the test set and 99.832% on the training set, demonstrating its exceptional performance in fraud detection and consistent generalization across both datasets. These high scores indicate that the balanced dataset and standardized features significantly enhanced the decision tree's effectiveness. The use of both Gini and Entropy criteria was intended to compare their abilities to capture different data patterns and optimize model performance; Gini offers computational speed and simplicity, while Entropy provides a more detailed information gain calculation, potentially excelling in learning complex patterns.
- ✓ Accuracy Scores: The close alignment between test (99.865%) and training (99.832%) accuracies confirms the model is not overfitting.

Confusion Matrix Results:

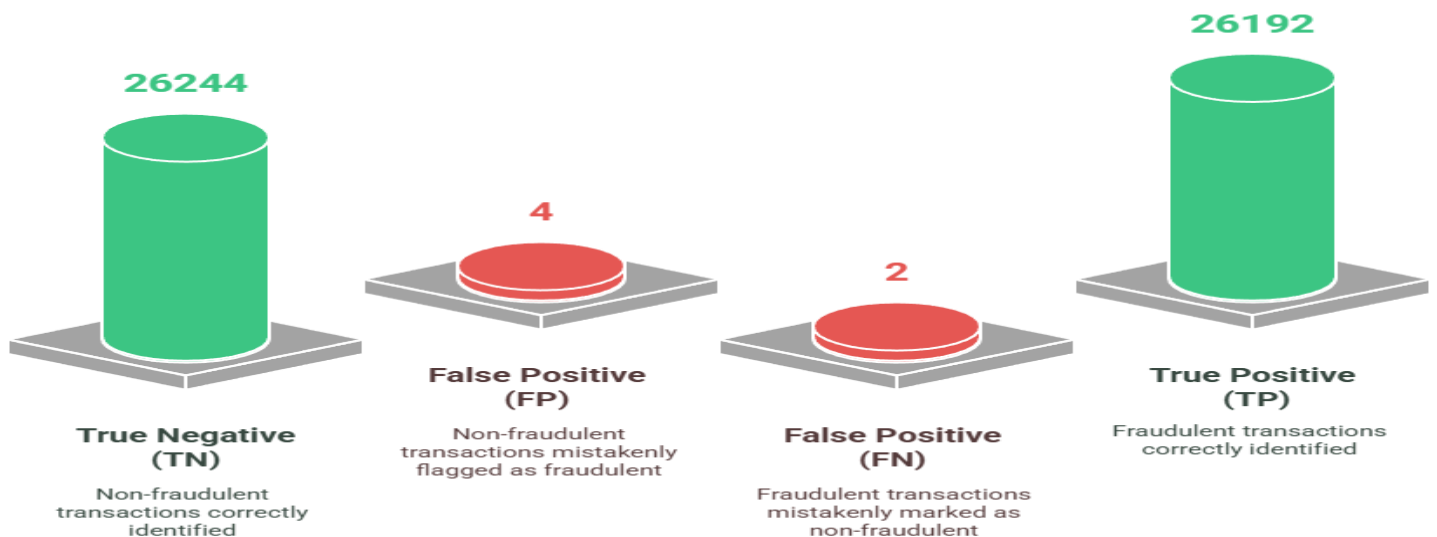
```
[ ] from sklearn.metrics import confusion_matrix, accuracy_score

    confusion_matrix(y_true=Y_test, y_pred= predicts)

⇒ array([[26244,    4],
        [    2, 26192]])
```

- ✓ Our model's confusion matrix shows it's doing an awesome job at spotting fraud with super tiny mistakes. It correctly nailed 26,244 non-fraud cases and 26,192 fraud cases, with just 4 non-fraud cases wrongly flagged as fraud and only 2 fraud cases missed, proving the balanced dataset and standardized features made the decision tree crazy effective.

Results from
Confusion Matrix



Made with Napkin

- ✚ True Negatives (TN = 26,244): The model got 26,244 non-fraud transactions right, showing it's great at identifying normal cases.
- ✚ True Positives (TP = 26,192): It correctly spotted 26,192 fraud cases, meaning it's super sharp at catching fraud.
- ✚ False Positives (FP = 4): Only 4 non-fraud cases were mistakenly tagged as fraud, so it's not causing many false alarms.
- ✚ False Negatives (FN = 2): Just 2 fraud cases were missed, which is awesome since missing fraud is a big deal.
- ✚ The tiny errors (4 FP, 2 FN) lead to a near-perfect accuracy (~99.989%), highlighting the model's strength on the balanced dataset.

What else could have been done?

1- Trying Different Models: Decision trees are pretty awesome, but fancier models like Random Forest or Gradient Boosting can catch trickier patterns in the data, which could cut down those pesky false positives (4) and false negatives (2) even more.

2- Tuning Hyperparameters: We used `max_depth=5` for our Gini model, but we didn't check if that's the best choice, so stuff like `max_depth` or `min_samples_split` could make the model smarter and reduce mistakes.

3-Focusing on Errors with Weighted Loss: Making the model more sensitive to false negatives by class weights we learned at other slides (Deep Learning, Neurons) can help catch more fraud and reduce errors.

CONCULISON

The model achieved an accuracy score of 99.865% on the test set and 99.832% on the training set, demonstrating its exceptional performance in fraud detection and consistent generalization across both datasets. These high scores indicate that the balanced dataset and standardized features significantly enhanced the decision tree's effectiveness. The use of both Gini and Entropy criteria was intended to compare their abilities to capture different data patterns and optimize model performance; Gini offers computational speed and simplicity, while Entropy provides a more detailed information gain calculation, potentially excelling in learning complex patterns.

